

CSE321 Study Questions Examples Solutions

1. (a) The DFS-based solution consists of following steps:

- Create the graph.
- Use a list to keep the track of visited vertices, initialize it with "False" values for each vertex.
- Initialize an empty list to keep the output sequence.
- Do the followings for each unvisited vertex.
 - Mark the vertex as visited.
 - Repeat this for all of the outgoing neighbors of this vertex.
 - Add the vertex to the output sequence.
- Reverse the list (because it is generated in the reverse order).

In this way, the algorithm visit all the vertices and creates a queue according to the neighborhood relations between them. If we assume there are v vertices and e edges, the algorithm will take $O(v + e)$ time to construct the sequence. Because it is analyzing each vertex and its outgoing neighbors (since outgoing neighbors are represented with edges, we can say that the algorithm is analyzing each vertex and each edge).

(b) A solution without using DFS approach is to benefit in-degrees of vertices. The algorithm given in .py file follows these steps:

- Create the graph.
- Generate a list of in-degrees of vertices, let's call it D .
- Generate a list of vertices with 0 in-degrees, let's call it Z .
- Initialize an empty list to keep the output sequence.
- Do the followings for each vertex from Z .
 - Add the vertex to the output sequence.
 - Delete the vertex from Z . Also, delete the edges that it is connected to from the edge list.
 - After the deletion of a vertex and some edges, update D .
 - According to D , update Z .

Since a DAG has at least one vertex with 0 in-degree, this algorithm benefits this information. At each step, a vertex with 0 in-degree is added to the output sequence. Because if a vertex has 0 in-degree, it means that it has no prerequisites. And when a vertex is added to the output sequence, we should delete it from the graph and update the in-degree list. Because we already solved that vertex's position in the queue and we can ignore it along with the edges it is connected to. Similarly, if we have v vertices and e edges, this algorithm has $O(v + e)$ time complexity as the one above. Because it checks every vertex in a loop. Within the loop it checks all of the outgoing neighbors of the related vertex. The number of total outgoing neighbors are equal to the number of edges. Therefore the algorithm goes over v vertices and e edges.

2. Logarithmic time indicates solving the problem by dividing it into x pieces, where x is the base of the logarithm. In this question, base is 2. Therefore, we should calculate the half of the problem at each step. Since $a^n = a^{(n/2)^2}$, it is enough to calculate $a^{(n/2)}$ and multiply it with itself instead of calculating a^n . Again, instead of calculating $a^{(n/2)}$, we can calculate $a^{(n/4)}$. This will go up to a^1 which is a itself. In this way, we make $\log_2 n$ calculations to compute a^n , thus the time complexity becomes $O(\log_2 n)$.

PS: If n is an odd number, we can do this operation for $a^{(n-1)}$ and then multiply the result with a since $a^n = a^{(n-1)} * a$.

3. The solution in the .py file follows these steps:

- Generate a puzzle.
- Do the followings for each cell, starting from the upper left one.
 - If the cell is out of the frame of the puzzle, this means that we have completed the solution, therefore return 'True'.
 - If the cell already has a value (the input values), continue with the next cell.
 - If the cell doesn't have a value, then try each value starting from 1 to 9, stop when found a valid solution for that cell. To check if a value is valid, do the followings.
 - * Analyze the 3x3 box of that cell, check if it is valid in 3x3 box.
 - * If it is valid in 3x3 box, then check if it is valid for 9x9 box.
 - * If both are true, then assign that value to that cell.

There are 9 possible solutions for each cell and we do these calculations for each cell. If we assume that the size of the puzzle is $N \times N$, then we would have $N \times N$ cells, therefore the time complexity would be $O(9^{(N \times N)})$. In this problem the time complexity is $O(9^{81})$.

4. There are several decrease-and-conquer algorithms for this problem. They are more subtle than one might expect. Generating combinations in a predefined order (increasing, decreasing, lexicographic) helps with both a design and a correctness proof. The following simple property is very helpful. Assuming with no loss of generality that the underlying set is $\{1, 2, \dots, n\}$, there are $\binom{n-i}{k-i}$ k -subsets whose smallest element is i , $i=1, 2, \dots, n-k+1$.

Here is a recursive algorithm from "Problems on Algorithms" by Ian Parberry [Par95], p.120:

```
Function choose(i, k)
//Generates all k-subsets of {i, i + 1, ..., n}
//stored in global array A[1..k]
// in descending order of their components
  if k = 0
    print(A)
  else
    for j = i to n - k + 1
      A[k] = j
      choose(j + 1, k - 1)
    end for
End Function
```

CSE321 Study Questions Examples Solutions

5. (a) Exhaustive search is a brute force solution used for combinatorial problems while brute force is usually used for searching a string pattern. Both are time consuming, but exhaustive search is known to be relatively faster. Unlike brute force, exhaustive search has a direction to follow at each step.
- (b) Caesar's Cipher is vulnerable to brute force attack because the key may have 26 different values (for English alphabet). Trying out each of them will end up with the solution of the cipher text. On the other hand, AES is not vulnerable to brute force attacks. AES makes sure the key is as long as to make it impossible to crack in a meaningful amount of time. For example, cracking a 256 bit AES key would take $\approx 2.29 * 10^{32}$ years, because the key space is too large.
- (c) While computing the complexity, we should take the size of the input into consideration. For instance, if the input is an array, we should check the number of elements in the array. In this problem, n is not the size, it is just a value. The size of n is calculated in the number of bits that presents it in base 2. To store n in base 2, we need $\log_2 n$ bits. Which makes the size of our input $n^{\log_2 n}$ which is exponential, not polynomial.

Why $n^{\log_2 n}$? We check the divisibility of n with $[2, 3, \dots, n-1]$. These numbers are represented as following:

- 2 is represented with $\lceil \log_2 2 \rceil = 1$ bit
- 3 is represented with $\lceil \log_2 3 \rceil = 2$ bits
- 4 is represented with $\lceil \log_2 4 \rceil = 2$ bits
- ...
- $n-1$ is represented with $\lceil \log_2 (n-1) \rceil$ bits.

As we see, we have $n-2$ numbers and each of them needs at most $\lceil \log_2 (n-1) \rceil$ bits. While calculating the time complexity, we ignore constant values, therefore we see that we will make calculations for n numbers, each requiring at most $\lceil \log_2 n \rceil$ bits. Thus, we will work on $n^{\log_2 n}$ bits.

Notes:

For the first 3 problems, please refer to the attached single .py file for the source codes.